Aprendizaje por refuerzo con verificación certificada como única recompensa: bisección bajo V_I con lemas y memoria

Informe de corrida del proyecto `rl_metodos`

Carlos Manuel Orrego Franco
Universidad Nacional de Colombia, Sede Manizales

Septiembre de 2026 — informe de la corrida `20260923-224900-gpt-6-astra-high`

1 Propósito

Esta nota fija el lenguaje común de los subproyectos de `rl_metodos`. Todos comparten una misma estructura: un *proponente* produce un objeto de prueba y un *juez* independiente decide si lo acepta. La recompensa proviene únicamente del juez; el proponente nunca se evalúa a sí mismo. Lo que varía entre subproyectos es el juez, y con él el residuo de confianza que la aceptación deja sin certificar. Esa variación es deliberada: es el experimento.

Esta corrida (`20260923-224900-gpt-6-astra-high`, en `runs/` bajo `bisection_isabelle_lemas_mem`) usa el juez V_I en su variante de programa (anotación de un bucle en Isabelle/ITrees) sobre el objetivo `isabelle:bisection` (hash `974945f4c4df07ee`), con el proponente `gpt-6-astra` y esfuerzo de razonamiento `high`. Presupuesto $B = 4$ intentos por episodio; estado: terminada; 3 episodios completos; el episodio 3 se repitió en la corrida `20260924-060102-gpt-6-astra-high`, que parte de la memoria del final del episodio 2 (la corrida original lo perdió: 4 intentos PROVIDER_ERROR, sin respuesta del modelo, que se apartan). Memoria entre episodios: sí (`run`; encabezado de la biblioteca en su redacción v1, que atribuye los lemas a Isabelle; la redacción v2 atribuye bien: el agente demuestra, Isabelle verifica. Se conserva v1 para comparar modelos con el mismo mensaje). Lemas auxiliares del agente: sí. Pregunta que responde: ¿basta que `gpt-6-astra` escriba lemas auxiliares propios, cuya demostración verifica el kernel antes de usarlos, para cerrar las obligaciones que la automatización de una línea deja abiertas? ¿Y ayuda recordar entre episodios los lemas ya verificados y el mejor intento?

Nota de la carpeta `20260923-224900-gpt-6-astra-high` (`LEEME.md`): «# Astra, bisección con lemas y memoria: el episodio 3 se repitió en otra carpeta Episodios 1 y 2 (8 intentos) son válidos. En el episodio 3 la API de OpenAI primero agotó el tiempo varias veces y después se quedó sin créditos (‘429 insufficient_quota‘, 24 sep 2026 ~04:44): sus 4 intentos son PROVIDER_ERROR, no evaluaciones del juez. Ese episodio se repitió el 24 sep desde las 06:01 en 20260924-060102-gpt-6-astra-high, que parte de la memoria del final del episodio 2 (logs/astra_ep3_memoria.json) y corre con el mismo código congelado. Los informes fusionan ambas carpetas (continuacion.json).»

Este informe sigue la estructura de la nota técnica: el MDP común (§2), la interfaz de verificador y sus tres instancias (§3), las funciones de recompensa (§4), las métricas (§5), la jerarquía de confianza entre jueces (§6) y los entornos (§7, Tabla 4); cierra con los experimentos realizados y previstos (§8), y reporta en §5 los valores medidos.

2 El MDP común

Definición 1 (Entorno de demostración verificada)**.** *Un entorno es una tupla* $\mathcal{E} = (T, \mathcal{A}, V, B)$ *donde* T *es un* objetivo *inmutable (un enunciado y su contexto),* $\mathcal{A}$ *es un espacio de* propuestas,

V es un verificador *y $B \in \mathbb{N}$ es un presupuesto de pasos. El MDP asociado es $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ con:*

- ***Estado*** *$s_t = (T, \mathcal{C}_t, h_t, M_t)$: el objetivo, el* contexto observable *$\mathcal{C}_t$ que el juez devuelve tras el último intento (lemas disponibles, obligaciones abiertas con su texto literal, o el valor certificado vigente), y el historial h_t de resultados anteriores.*
- ***Memoria*** *M_t: biblioteca de lemas que el agente demostró y que Isabelle verificó (se añaden solos a cada intento) y resumen del mejor intento de cada episodio anterior. Persiste entre episodios de la corrida. El modelo está congelado: la memoria es contexto, no entrenamiento.*
- ***Acción*** *$a_t \in \mathcal{A}$: una propuesta completa (una cadena de lemas, una anotación de invariantes y variantes, o un programa de tamaños de paso). El proponente es un modelo de lenguaje; la acción es su salida tras una extracción determinista.*
- ***Transición*** *$P(s_{t+1} \mid s_t, a_t)$: determinista dado el juez; s_{t+1} incorpora el resultado $V(T, a_t)$ y actualiza $\mathcal{C}_{t+1}$ y h_{t+1}. La estocasticidad del sistema vive en el proponente, no en la transición.*
- ***Recompensa*** *$R(s_t, a_t) = r(V(T, a_t))$, función únicamente del resultado del juez (§4).*
- ***Terminación****: el episodio termina en el primer t con resultado* Accepted*, o cuando $t = B$.*

Observación 1 (El objetivo es inmutable)**.** *T no forma parte del espacio de acciones. Cualquier propuesta que modifique el enunciado, sus hipótesis o su tipo se rechaza con etiqueta* Statement_Drift *(§4.3). Esta restricción se impone en código mediante un hash del enunciado congelado, no por convención. Es la* frontera semántica *del proyecto: el lugar donde la confianza no puede delegarse al juez. En esta corrida, el hash del objetivo es `974945f4c4df07ee`. En V_I el hash cubre las importaciones, la matemática auxiliar, la plantilla del programa, las hipótesis y la conclusión; se recalcula desde disco en cada verificación.*

3 La interfaz de verificador y sus tres instancias

Definición 2 (Verificador)**.** *Un verificador es una función $V : \mathcal{T} \times \mathcal{A} \to \mathcal{O}$ que devuelve un* resultado

$$o = (\texttt{accepted},\ \texttt{label},\ \texttt{partial},\ \texttt{cost_s},\ \texttt{raw}),$$

con $\texttt{accepted} \in \{0, 1\}$, $\texttt{label} \in L$ y $\texttt{partial} \in [0, 1]$, donde L es un conjunto finito de etiquetas de resultado propio de cada juez, $\texttt{partial}$ es una medida de progreso que sólo se usa bajo recompensa moldeada, y $\texttt{raw}$ es la salida íntegra del juez para auditoría.

3.1 Juez de grafo de lemas (V_G)

Proviene de los subproyectos 01–05. Un objetivo T es un grafo dirigido acíclico $G = (\Lambda, D)$ de pasos $\lambda \in \Lambda$, cada uno con dependencias $D(\lambda) \subseteq \Lambda$, un subconjunto $\Delta \subset \Lambda$ de *distractores* (pasos falsos o inaplicables, tomados de errores reales de estudiantes) y un paso distinguido Qed. Una propuesta es una secuencia $a = (\lambda_1, \ldots, \lambda_k)$ de identificadores anónimos (`L01`, `L02`, ..., barajados con semilla fija); no se muestran los títulos de los pasos ni las refutaciones del profesor, que delatan al distractor. El juez acepta si existe $j \le k$ con $\lambda_j =$ Qed tal que, para todo $i \le j$,

$$\lambda_i \notin \Delta, \quad \lambda_i \notin \{\lambda_1, \ldots, \lambda_{i-1}\}, \quad D(\lambda_i) \subseteq \{\lambda_1, \ldots, \lambda_{i-1}\};$$

lo que venga después de Qed se ignora. Etiquetas, asignadas en el primer paso que falla: Accepted (la cadena llega a Qed), Invalid_Step (dependencia no establecida o paso repetido), Distractor (el paso es un distractor), Unknown_Lemma (identificador ajeno al grafo) y Incomplete (todos los pasos son válidos pero la cadena no llega a Qed; es un resultado por intento, no por presupuesto). Progreso parcial: fracción del cierre de dependencias de Qed establecida antes del primer fallo.

3.2 Juez PEP (V_P)

Proviene del subproyecto 06, que se usa sin modificaciones. El objetivo es una instancia: la clase de funciones L-suaves (μ-fuertemente) convexas con $L = 1$, un horizonte n, un criterio ($f(x_n) - f^*$ o $\|\nabla f(x_n)\|^2$, con $\|x_0 - x^*\| \leq 1$) y si se admiten pasos negativos; una propuesta es un programa `schedule(n)` que genera pasos $(h_0, \ldots, h_{n-1})$. El programa se ejecuta en un intérprete aparte (en macOS, bajo un *sandbox* sin red ni escritura en disco) y el juez puntúa, en un proceso que el programa no tocó, la sucesión literal que devuelve; las referencias se leen una sola vez al empezar. El juez resuelve el problema de estimación de desempeño (PEP) como un programa semidefinido y obtiene el peor caso $\tau(h)$; el certificado dual es la demostración de la cota (Drori and Teboulle, 2014; Taylor et al., 2017). En el modo por defecto, `improve`, toda mejora afirmada se recalcula con tolerancias finas y se certifica en aritmética racional exacta; se acepta sólo si la cota certificada es estrictamente menor que τ_{ref}. La referencia τ_{ref} es el mínimo entre la tabla publicada de óptimos numéricos, que sólo cubre $f(x_n) - f^*$ con $\mu = 0$ (Das Gupta et al., 2024; Grimmer et al., 2024), y la búsqueda propia de 06 por entropía cruzada; en las variantes abiertas (norma del gradiente, fuertemente convexo, pasos negativos) sólo hay búsqueda propia. Etiquetas: CERTIFIED (mejora estricta de τ_{ref} con certificado racional exacto; es la aceptación), MATCHED (sólo en modo `match`: iguala τ_{ref} con tolerancia relativa 10^{-4}, sin certificado exacto), INVALID_SCHEDULE (el programa falla, se agota su tiempo o no devuelve n números finitos válidos), NOT_IMPROVED (programa válido que no mejora τ_{ref}) y CERTIFICATION_FAILED (mejora numérica cuyo certificado exacto falla).

3.3 Juez Isabelle (V_I)

Proviene del subproyecto 08 y del marco Isabelle/ITrees (Bryant et al., 2026). Un objetivo es un programa imperativo junto con un triple de Hoare de corrección total $H[P]\,C\,[Q]$, ambos congelados; una propuesta es una anotación $a = (inv, var)$ de invariante y variante de bucle y, en fase 2, además lemas auxiliares con su demostración, escritos por el agente y verificados en el kernel antes de usarse. El juez procede por etapas: (0) la anotación pasa una lista blanca de símbolos y el kernel demuestra que el programa anotado es igual al congelado con anotaciones fijas; (0b) si hay lemas, pasan un filtro léxico (sólo enunciados con su demostración: nada de `sorry`, definiciones, notación ni ML) y se compilan en el contexto de las hipótesis, comprobando que el enunciado congelado se lee igual y que no aparecen axiomas nuevos; (1) inserta la anotación y ejecuta el generador de condiciones de verificación `vcg`; (2) intenta cerrar cada obligación con automatización de una línea (`try0` y, si falla, Sledgehammer con tiempo límite fijo), y sólo cuenta una sugerencia de Sledgehammer si su reconstrucción (*preplay*) terminó; (3) si todas cierran, ensambla el teorema completo, lo compila y lo traslada al programa congelado; (4) comprueba que no dependa de oráculos, de `skip_proof` ni de axiomas nuevos. Etiquetas: ACCEPTED (todas las etapas pasan: el teorema está en el kernel), VC_OPEN (alguna obligación sin prueba de una línea), VCG_FAILED (la anotación no se analiza o no tipa, o `vcg` falla), LEMMA_FAILED (el kernel rechaza la demostración de algún lema del agente), ASSEMBLY_FAILED (las obligaciones se cierran por separado pero el teorema ensamblado no compila), UNSOUND (`sorry`, `oops`, oráculos, `skip_proof` o axiomas nuevos), STATEMENT_DRIFT (la propuesta sale de su hueco o el kernel no prueba que el programa anotado sea el congelado), TIMEOUT (se agota el tiempo de compilación) y INVALID_PROPOSAL (falta el invariante o el variante, o hay símbolos o comandos no admitidos). Progreso parcial: fracción de obligaciones cerradas en VC_OPEN y ASSEMBLY_FAILED, y 0 en las demás.

En esta corrida. Objetivo `isabelle:bisection` (hash `974945f4c4df07ee`); lemas prestados: `Bolzanos_theorem`, `sign_prop`. `try0`: 5 s; Sledgehammer: 60 s por obligación; límite de compilación: 1800 s. Lemas del agente: sí; memoria: sí. Código de `unirl`: huella `61c5c2b8649611f6`, commit `dcb8f7b` con cambios sin versionar.

Observación 2 (Existencia frente a descubrimiento)**.** *Con la anotación del propio artículo (su*

invariante y su variante), V_I genera 15 obligaciones y la automatización de una línea cierra 9 en esta máquina, con Sledgehammer limitado a 60 s por obligación: VC_Open, 424 s de juez (`runs/_baseline/bisection.json`). Quedan abiertas 4 de variante, 2 de postcondición (clasificación heurística por la conclusión y las premisas de cada obligación). Adelgazar las premisas no basta: sobre una obligación del variante, con todas sus premisas o con una sola, y con o sin los hechos previos de la demostración del artículo, Sledgehammer encuentra prueba en 0 de 4 ensayos de 60 s (`sh_goals.json`). Con la misma anotación y, como lemas del agente, las demostraciones manuales de los autores (los hechos previos, la línea `smt` del variante y `bisect_root_extract`), el juez da Accepted: 15 de 15 obligaciones cerradas, 5 lemas verificados en el kernel, de los que las pruebas de una línea usan `a_less_than_b`, `bisect_root_extract`, `var_dec`; 214 s de juez (`upper_bound.json`). Existe, pues, una demostración que V_I acepta; lo que la automatización no descubre es justamente la parte que los autores escribieron a mano. Por eso V_I distingue tres líneas base: automatización sobre todas las premisas de `vcg`, automatización con un filtro de relevancia estándar, y selección de premisas como acción explícita del agente. Nunca se mezclan. Hoy están implementadas la primera y la acción de lemas auxiliares del agente; el filtro de relevancia y la selección de premisas son trabajo futuro.

4 Funciones de recompensa

4.1 Recompensa base (todos los jueces)

$$r(o) = \begin{cases} 1 & \text{si } o.\texttt{accepted} = 1, \\ 0 & \text{en otro caso.} \end{cases}$$

Es la única recompensa activa por defecto. Es binaria a propósito: cualquier señal intermedia introduce un juicio sobre qué es "progreso", y ese juicio debe declararse, no esconderse.

4.2 Recompensa moldeada (opcional, tras bandera)

$$r_\beta(o) = r(o) + \beta \cdot o.\texttt{partial} \cdot \mathbb{1}[o.\texttt{accepted} = 0], \qquad \beta \in [0, 0.5].$$

Para V_P se conserva la recompensa original del subproyecto 06, que es continua por naturaleza: $r_P(o) = \max\{-2,\ \log(\tau_{\text{ref}}/\tau(h))\}$ si el programa es válido y -3 en otro caso, donde τ_{ref} es la mejor cota certificada conocida. Se reporta también su binarización $\mathbb{1}[\tau(h) < \tau_{\text{ref}}]$ para hacerla comparable con los otros jueces.

4.3 Penalización por deriva del enunciado

Statement_Drift recibe recompensa 0, no negativa, en la configuración base. La razón es metodológica: queremos *contar* cuántas veces el proponente intenta cambiar la pregunta, y una penalización alteraría la frecuencia que se pretende medir. Una variante con penalización $-\delta$ queda disponible tras bandera para experimentos de control.

En esta corrida: recompensa base binaria $r(o)$; la moldeada está desactivada ($\beta = 0$); penalización de deriva $\delta = 0$ (Statement_Drift recibe 0). En esta corrida el proponente no aprende de la recompensa: el modelo está congelado y la recompensa se registra.

5 Métricas

Para cada entorno, juez y proponente se registran:

1. **Intentos hasta aceptación**: mediana y distribución de t^* sobre episodios, con $t^* = B$ si no hay aceptación.

En esta corrida: t^* mediana 4 sobre 3 episodios completos, censurada en $B = 4$; por episodio: ep. 1: 4 (sin aceptar), ep. 2: 4 (sin aceptar), ep. 3: 4 (sin aceptar); aceptados 0 de 3.

2. **Distribución de etiquetas**: conteo de cada $\ell \in L$ sobre todos los intentos.

   En esta corrida: Tabla 1: 12 intentos en total.

3. **Razón de deriva**: para V_I, el cociente entre el número de resultados STATEMENT_DRIFT y el de VC_OPEN; para V_G, el cociente entre DISTRACTOR e INVALID_STEP. Mide cuán a menudo el proponente "resuelve" cambiando la pregunta o cayendo en un error de estudiante.

   En esta corrida: STATEMENT_DRIFT/VC_OPEN = 0/0.

4. **Transferencia**: reducción de t^* en el objetivo T_2 cuando el proponente ha sido entrenado (o condicionado con ejemplos aceptados) sobre T_1, frente a partir de cero.

   En esta corrida: No aplica: la memoria de esta corrida recuerda entre episodios del *mismo* objetivo; no es transferencia entre objetivos, y no se midió ninguna.

5. **Comparación entre jueces**: para un mismo método y un mismo proponente, t^* y etiquetas bajo V_G frente a V_I. Es la medida directa de cuánto de la "facilidad" de un juez proviene del trabajo humano incorporado en él.

   En esta corrida: Tabla 2. Los enunciados de V_G y V_I no coinciden, así que la comparación es indirecta.

6. **Costo**: segundos de juez por intento; determina el cuello de botella real.

   En esta corrida: Segundos de juez por intento: media 35,4, mediana 28,1, máximo 116,7; segundos de modelo: media 1960,7, mediana 1490,1, máximo 8922,3. Suma: 424 s de juez y 23529 s de modelo. Duración desde el inicio hasta el último intento registrado: 8 h 28 min (desde el inicio de la corrida original hasta el último intento de la continuación; incluye la pausa entre ambas carpetas).

etiqueta	intentos
LEMMA_FAILED	12
total	12

Table 1: Etiquetas del juez sobre todos los intentos de la corrida.

entorno	juez	proponente	ep.	acept.	B	t^* med.	intentos	s juez	s LLM
`bisection_isabelle`	V_I	`gpt-6-luna` (high)	3	0	4	4	12	648,4	72,4
`bisection_isabelle_lemas`	V_I	`gpt-6-luna` (high)	1	0	4	4	4	411,0	95,8
`bisection_isabelle_lemas_mem`*	V_I	`gpt-6-astra` (high)	3	0	4	4	12	35,4	1960,7
`bisection_isabelle_lemas_mem`	V_I	`gpt-6-luna` (high)	3	0	4	4	12	179,5	87,4
`bisection_graph`	V_G	`gpt-6-luna` (high)	20	20	4	1	20	0,0	8,4

Table 2: Corridas del mismo método (bisección) en `runs/` (se ignoran las carpetas que empiezan por "_"), agrupadas por entorno, juez y proponente como en `python -m unirl summary`. "ep." cuenta episodios completos; los segundos son medias por intento evaluado; "intentos" no cuenta los PROVIDER_ERROR (sin respuesta del modelo). *: entornos y proponente de este informe.

Biblioteca de la memoria al generar el informe (0 lemas, que `gpt-6-astra` demostró y cuya demostración Isabelle verificó): vacía.

Primer error de Isabelle (o motivo del juez) en los intentos que no llegaron a las obligaciones, saneado y recortado:

ep.	paso	etiqueta	VCs cerradas	abiertas (tipo)	lemas prop.	lemas verif.	s juez	s LLM
1	1	LEMMA_FAILED	—	—	6	0	28	2161
1	2	LEMMA_FAILED	—	—	4	0	29	758
1	3	LEMMA_FAILED	—	—	7	0	27	812
1	4	LEMMA_FAILED	—	—	8	0	27	2096
2	1	LEMMA_FAILED	—	—	6	0	28	2192
2	2	LEMMA_FAILED	—	—	8	0	28	8922
2	3	LEMMA_FAILED	—	—	6	0	28	1487
2	4	LEMMA_FAILED	—	—	11	0	27	2096
3	1	LEMMA_FAILED	—	—	8	0	29	899
3	2	LEMMA_FAILED	—	—	9	0	28	278
3	3	LEMMA_FAILED	—	—	9	0	117	1493
3	4	LEMMA_FAILED	—	—	11	0	28	334

Table 3: Intentos en orden. "VCs cerradas" cuenta las obligaciones con prueba de una línea; el tipo de cada VC abierta (invariante, variante, postcondición, inicialización) se asigna por su conclusión y sus premisas (heurística de `unirl/report.py`). "Lemas prop." son los que el agente escribió en ese intento; "lemas verif.", los que el kernel verificó en ese intento, incluidos los de la biblioteca de la memoria; — indica que el registro de la corrida no lo guarda.

- ep. 1, paso 1, LEMMA_FAILED: Failed to finish proof (línea 102): goal (1 subgoal): 1. [|0 < (b - a) / tol|] ==> (real k < log 2 ((b - a) / tol)) =
- ep. 1, paso 2, LEMMA_FAILED: Failed to apply initial proof method (línea 128): using this: int k < ceiling(log 2 ((b - a) / tol)) goal (1 subgoal):
- ep. 1, paso 3, LEMMA_FAILED: Failed to finish proof (línea 142): goal (1 subgoal): 1. [|(if 1 + int k <= n then n - (1 + int k) else 0) < (if int k <= n then n - int k else 0)|]
- ep. 1, paso 4, LEMMA_FAILED: Failed to finish proof (línea 132): goal (1 subgoal): 1. int k <= n
- ep. 2, paso 1, LEMMA_FAILED: Failed to apply initial proof method (línea 133): using this: q - p = (b - a) / 2 ^ k (b - a) / 2 ^ (k + 1) = (b - a) / 2 ^ k / 2
- ep. 2, paso 2, LEMMA_FAILED: Failed to finish proof (línea 147): goal (1 subgoal): 1. [|(if 1 + int k <= n then n - (1 + int k) else 0) < (if int k <= n then n - int k else 0)|]
- ep. 2, paso 3, LEMMA_FAILED: Failed to apply initial proof method (línea 133): goal (1 subgoal): 1. (p + q) / 2 - p = (q - p) /
- ep. 2, paso 4, LEMMA_FAILED: Undefined method: "field_simp" (línea 156)
- ep. 3, paso 1, LEMMA_FAILED: Failed to finish proof (línea 102): goal (1 subgoal): 1. [|0 < (b - a) / tol|] ==> (2 ^ k < (b - a) / tol) =
- ep. 3, paso 2, LEMMA_FAILED: Outer syntax error (línea 123): proposition expected, but end-of-input (línea 123) was found
- ep. 3, paso 3, LEMMA_FAILED: exception Interrupt_Breakdown raised
- ep. 3, paso 4, LEMMA_FAILED: Failed to finish proof (línea 156): goal (1 subgoal): 1. [|(if 1 + int k <= n then n - (1 + int k) else 0) < (if int k <= n then n - int k else 0)|]

6 Jerarquía de confianza entre jueces

Los tres jueces aceptan objetos distintos y dejan residuos de confianza distintos. Ordenados de menor a mayor independencia respecto del humano:

juez	qué certifica	en qué reposa la confianza
V_G	validez de una cadena en un grafo dado	el grafo, escrito por el profesor
V_P	una cota numérica de peor caso	el SDP, la certificación racional exacta
V_I	un teorema en el kernel de Isabelle/HOL	el kernel; la adecuación del triple congelado

Ningún juez elimina la confianza; cada uno la desplaza. V_G la desplaza al autor del grafo. V_P la desplaza a la formulación del PEP y a la implementación del certificador. V_I la desplaza a dos lugares que el kernel no puede inspeccionar: la correspondencia entre el triple de Hoare y el enunciado informal que se pretendía probar, y la matemática auxiliar importada. La Observación 2 muestra que incluso lo que el artículo de referencia reporta como "automático" contiene decisiones humanas en esa frontera. El objetivo del proyecto es medir ese residuo, no negarlo.

Residuo en esta corrida: los lemas prestados (`Bolzanos_theorem`, `sign_prop`), copiados con sus demostraciones: el kernel comprueba esas demostraciones, pero su elección es humana; la adecuación de la tripleta congelada al enunciado informal de la bisección; y el modelo del programa en números reales, no en punto flotante. Los lemas del agente no añaden residuo de corrección, porque el kernel verifica sus demostraciones antes de usarlos; sí lo añaden a la interpretación: el juez no decide si un lema "auxiliar" ya contiene la prueba pedida, como muestra la cota superior de la Observación 2.

7 Entornos

sub.	método	objetivo de fase 2	juez	lemas	distr.	cad. mín.
01	bisección	$m_n \to c$, $f(c) = 0$, error $\leq (b-a)/2^{n+1}$	V_G, V_I	21	7	14
02	punto fijo	Banach en $[a, b]$: unicidad, convergencia, cotas	V_G	20	7	13
03	Taylor	resto de Lagrange; convergencia con derivadas acotadas	V_G	20	7	13
04	Newton	convergencia local cuadrática	V_G	18	7	11
05	secante	convergencia local de orden φ	V_G	21	8	13
06	pasos de GD	cota de peor caso $f(x_n) - f^*$ o $\|\nabla f(x_n)\|^2$ (PEP)	V_P	—	—	—
08*	bisección	triple de Hoare de corrección total del programa `bisection`	V_I (programa)	—	—	—
09	pasos de GD	4 teoremas de cota: `gd_const_dt`, `gd_const_n2`, `gd_const_textbook`, `gd_n1`	V_I (teorema)	—	—	—

Table 4: Entornos de `unirl`. En 01–05 los números se recalculan de los grafos (`proof_kb`): "lemas" cuenta todos los pasos, incluidos distractores y QED; "cad. mín." es el cierre de dependencias de QED. Objetivos de V_I disponibles: `bisection` (variante de programa) y 4 cotas de GD (variante de teorema). El punto fijo aún no tiene objetivo formal; Taylor, Newton y secante sólo disponen de juez de grafo. Proponentes: `gpt-6-luna` y `gpt-6-astra`, ambos con esfuerzo `high`, para todos los jueces (corridas en `runs/`: `gpt-6-luna`: 11, `gpt-6-astra`: 7). *: filas de este informe.

8 Experimentos realizados y previstos

1. *Este informe*: `bisection_isabelle_lemas_mem` con `gpt-6-astra` (high) (`20260923-224900-gpt-6-astra-high`; terminada; 3 episodios completos; el episodio 3 se repitió en la corrida `20260924-060102-gpt-6-astra-high`, que parte de la memoria del final del episodio 2 (la corrida original lo perdió: 4 intentos PROVIDER_ERROR, sin respuesta del

modelo, que se apartan)). 0 aceptados de 3 episodios completos, t^* mediana 4 ($B = 4$); etiquetas: LEMMA_FAILED 12; biblioteca final de 0 lemas verificados.

2. Relacionada: `bisection_graph` bajo V_G con `gpt-6-luna` (high) (`20260923-193740-gpt-6-luna-high`). 20 aceptados de 20 episodios completos, t^* mediana 1 ($B = 4$); etiquetas: ACCEPTED 20.

3. Relacionada: `bisection_isabelle` bajo V_I con `gpt-6-luna` (high) (`20260923-155924-gpt-6-luna-high`). 0 aceptados de 3 episodios completos, t^* mediana 4 ($B = 4$); etiquetas: VC_OPEN 12; mejor intento 15/25 obligaciones cerradas.

4. Relacionada: `bisection_isabelle_lemas` bajo V_I con `gpt-6-luna` (high) (`20260923-211101-gpt-6-luna-high`; **detenida** antes de terminar; 1 episodio completo). 0 aceptados de 1 episodio completo, t^* mediana 4 ($B = 4$); etiquetas: VC_OPEN 2, LEMMA_FAILED 2; mejor intento 7/12 obligaciones cerradas.

5. Relacionada: `bisection_isabelle_lemas_mem` bajo V_I con `gpt-6-luna` (high) (`20260923-215521-gpt-6-luna-high`). 0 aceptados de 3 episodios completos, t^* mediana 4 ($B = 4$); etiquetas: LEMMA_FAILED 10, VC_OPEN 2; mejor intento 7/10 obligaciones cerradas; biblioteca final de 4 lemas verificados.

6. Prevista: transferencia entre objetivos (métrica 4), no implementada.

Observación 3 (Interpretación)**.** *Esta interpretación la redactó Claude a partir de los registros de la corrida; no es del autor.*

Astra no llegó nunca a las obligaciones: en los 12 intentos evaluados (episodios 1 y 2 de la corrida original y el episodio 3 repetido) Isabelle rechazó algún lema, y ningún lema quedó verificado, así que la biblioteca de la memoria terminó vacía. La estrategia fue opuesta a la de Luna: en cada intento Astra escribió entre 4 y 11 lemas, casi la demostración completa del teorema partida en piezas (cota del contador, paso del variante, mitad del ancho, signo en el subintervalo, raíz en el interior), en lugar de avanzar lema por lema. Como el juez exige que todos los lemas del bloque se verifiquen antes de mirar las obligaciones, un solo error bastó para perder cada intento. Los primeros errores fueron, sobre todo, demostraciones que no cierran (6) o cuyo método inicial no aplica (3); una vez usó `field_simp`, método de otro asistente de prueba, como Luna.

Las llamadas a Astra fueron muy lentas: entre 5 minutos y dos horas y media por intento, parte por tiempos agotados de la API. En el episodio 3 original la cuenta de OpenAI se quedó sin créditos; ese episodio se repitió desde la memoria del final del episodio 2 con el mismo código congelado, y sus 4 intentos sin respuesta del modelo se apartan de todas las métricas.

Frente a Luna con el mismo juez, la misma configuración y el mismo mensaje de memoria, ninguno de los dos aceptó un episodio. Luna verificó 4 lemas y llegó dos veces a las obligaciones; Astra, ninguna. En este objetivo la diferencia entre modelos no está en la matemática del enunciado sino en cuánto arriesga cada intento frente a un juez que no perdona un lema fallido.

References

Bryant, D., Huerta y Munive, J. J., and Foster, S. (2026). Verifying numerical methods with Isabelle/HOL. v3, 21 Sep 2026; código: github.com/isabelle-utp/Verified_Numerical_Methods.

Das Gupta, S., Van Parys, B. P. G., and Ryu, E. K. (2024). Branch-and-bound performance estimation programming: A unified methodology for constructing optimal optimization methods. Mathematical Programming.

Drori, Y. and Teboulle, M. (2014). Performance of first-order methods for smooth convex minimization: A novel approach. *Mathematical Programming*, 145(1–2):451–482.

Grimmer, B., Shu, K., and Wang, A. L. (2024). Composing optimized stepsize schedules for gradient descent.

Taylor, A. B., Hendrickx, J. M., and Glineur, F. (2017). Smooth strongly convex interpolation and exact worst-case performance of first-order methods. *Mathematical Programming*, 161(1–2):307–345.